

AMF-Placer

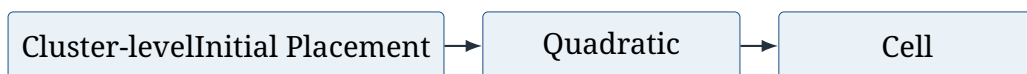
High-Performance Analytical Mixed-size Placer for FPGA

Day 2: Initial Placement 与 Quadratic Placement

从“先找到一个好起点”到“用二次模型稳定地压低线长”

今天只学两个阶段，但会学得很细：

- 为什么 AMF-Placer 不从随机 placement 开始，而先做 cluster-level SA；
- SA 的 cluster、bin、cost function 分别在表达什么；
- 为什么 HPWL 不能直接交给标准二次求解器；
- Bound2Bound quadratic model 和 pseudo net 在做什么；
- 为什么论文把 pseudo-net weight 从 $\alpha/\text{movement}$ 改成 $\alpha \text{ pinNum}/\text{movement}$ 。



目录

1 先把今天的两个阶段放回全流程

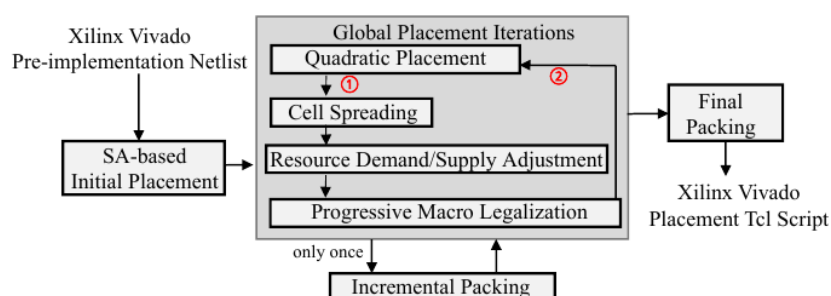


图 1 原论文 Fig. 3: AMF-Placer 的 7 阶段流程。Day 2 只聚焦最前面的 Initial Placement 与 Quadratic Placement。

Day 2 的主线可以压缩成一句话：

先用 SA 找到一个“不太差”的全局起点 → 再用 QP 高效地做连续坐标线长优化

两者分工不同：SA 强在跳出局部最优，QP 强在快速、可并行、连续优化。

AMF-Placer 把二次布局产生的结果称为 **lower-bound placement**：它偏向低 HPWL，但不一定已经满足所有离散资源与宏合法化约束；经过 spreading、legalization 等阶段得到的状态称为 **upper-bound placement**。当 upper/lower 两者逐渐接近，并且 macro 已经精确合法化，global placement 才趋于收敛。（论文 Sec. II-C）

不要把“lower bound”理解成数学上对全局最优 HPWL 的严格下界。论文这里主要是流程术语：**QP 解更自由、HPWL 更低，因此称 lower-bound placement**；约束更强的 placement 线长通常更高，因此称 **upper-bound placement**。

2 为什么 AMF-Placer 不从随机位置直接开始？

2.1 传统 analytical placer 的一个常见假设

论文指出，以前不少 FPGA analytical placers 直接随机初始化，因为常见经验是：分析式优化对初始位置“不太敏感”。但这些经验多来自随机生成 benchmark 或规模较小的设计。（论文 Sec. III-A）

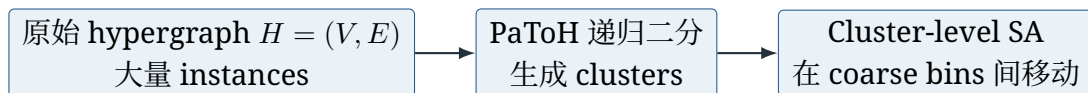
现代 FPGA 上，这个假设变得危险，原因是：

- 不同资源并不是均匀铺满整个芯片；
- DSP、BRAM、SLICEM 等往往集中在特定列或区域；
- macro 还带有 shape/resource constraints；
- 连续优化模型因此更容易出现明显的 local energy minima。

设想一个 BRAM-heavy cluster 一开始被扔在离 BRAM column 很远的位置。后面的 QP 虽然知道“线长往哪边下降”，却未必容易在复杂的资源约束下跨越一个很大的坏区域。初始 placement 的作用不是求最终答案，而是先把大尺度结构摆对大致方向。

2.2 AMF-Placer 的策略：先粗粒度，再细粒度

它不直接对几十万 instances 做 SA。那样太慢。论文先把 netlist 递归二分成较粗的 clusters，再只对 clusters 做 SA：



这是一种典型的 **coarse-to-fine** 思想：先在粗粒度层面解决“大的拓扑关系和资源方向”，再把细粒度自由度交给后续 analytical placement。

3 Step 1: 先把 netlist 划成 clusters

论文定义 cluster 集合

$$C = \{c_1, c_2, \dots, c_n\},$$

每个 cluster 最多包含 N_{Csize} 个 standard cells；macro 内部的 standard cells 也计入 cluster 大小。聚类采用 PaToH 递归 bi-partitioning。论文称其复杂度近似为 $O(m)$ ，其中 m 是 hyperedge/net 的数量。（论文 Sec. III-A）

3.1 为什么用 hypergraph partitioning?

在 netlist 中，一个 net 往往连接多个 pins，因此更自然的模型是 hypergraph：

$$H = (V, E),$$

其中 instance 是 vertex，net 是 hyperedge。partitioning 希望把连接紧密的 instances 尽量留在同一 cluster，减少跨 cluster 的连接。

可以把它直观地看成：如果一群 LUT/FF 彼此有很多内部连接，那么把它们作为一个 cluster 后一起粗移动，可以较好地保留设计层次；而跨 cluster 的 nets 才真正决定 clusters 之间应该靠多近。

3.2 不仅看连线，还要看资源和 clock

AMF-Placer 还对 cluster 加资源约束，例如限制 cluster 内 DSP 数量，避免一个 cluster 本身就超过某个区域可提供的资源。论文同时采用 clock-aware partitioning criteria，使 initial placement 对 clock region 更友好。（论文 Sec. III-A）

因此这里的 cluster 不是“只看图连接的纯图论分块”，而是：

connectivity + resource feasibility + clock awareness

这已经在 SA 之前加入了 FPGA 架构信息。

4 Step 2: 把 FPGA 粗分成 8×5 个 bins

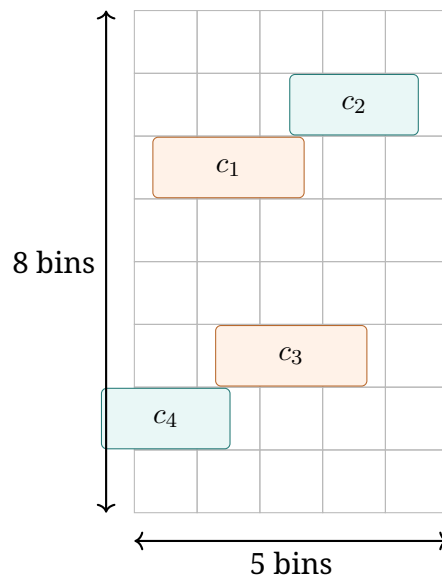
聚类完成后，论文把 FPGA device 均匀切成

$$N_{CY} \times N_{CX}$$

个 coarse bins，并经验性设置

$$N_{CY} = 8, \quad N_{CX} = 5.$$

随后 SA 的“状态”就不再是每个 cell 的精确坐标，而是“每个 cluster 被分配到哪个 coarse bin”。（论文 Sec. III-A）



论文为了快速估算 cost，假定 cluster 中所有 instances 暂时位于该 bin 的中心。这样 cluster 一旦换 bin，线长估计和密度/拥挤代价就能快速更新。

4.1 SA 到底在做什么？

它不断随机执行“cluster assignment / swap”，并依据 simulated annealing 的接受准则决定是否保留新状态。论文没有展开温度 schedule 的细节；这里最重要的是理解 SA 的功能：

它允许暂时接受变差的 **move**。因此 cluster 有机会跨过局部能量障碍，移动到更合理的资源区域或拓扑位置；这正是纯局部/梯度式方法不擅长的部分。

5 SA 的 cost function：线长 + 拥挤代价

论文给出

$$F_{cost} = \sum_{e \in E} W_e + \sum_{BC_{ij} \in B_C} Of(BC_{ij}), \quad (1)$$

其中 W_e 是 net e 的 HPWL， $Of(BC_{ij})$ 是 coarse bin 的 overflow/拥挤代价。（论文 Eq. (1)）

进一步：

$$Of(BC_{ij}) = \frac{(W_B + H_B) \left(\sum_{c \in BC_{ij}} \text{size}(c) \right)^2}{N_{Csize}}, \quad (2)$$

W_B, H_B 分别是 cluster bin 的宽与高。（论文 Eq. (2)）

5.1 先不要急着算，先看结构

$$\underbrace{F_{cost}}_{\text{SA 要最小化}} = \underbrace{\text{wirelength}}_{\text{clusters 要彼此靠近}} + \underbrace{\text{bin penalty}}_{\text{不要把太多资源塞进同一区域}}.$$

最关键的是平方项：

$$\left(\sum \text{size}(c) \right)^2.$$

当同一个 bin 中 cluster 总规模继续增加时，惩罚不是线性增长，而是更快地增长，从而把过度聚集的 clusters 推开。

论文原意

论文把该项称为 **resource overflow function**，但正文没有在这里进一步给出“按不同资源类型拆分”的公式。学习时不要擅自把 Eq. (2) 改写成某种标准的 $\max(0, \text{demand} - \text{supply})$ 形式；本笔记保持论文给出的表达式，只把它理解为 cluster-level 的拥挤/overflow 代价。

5.2 一个极简直觉例子

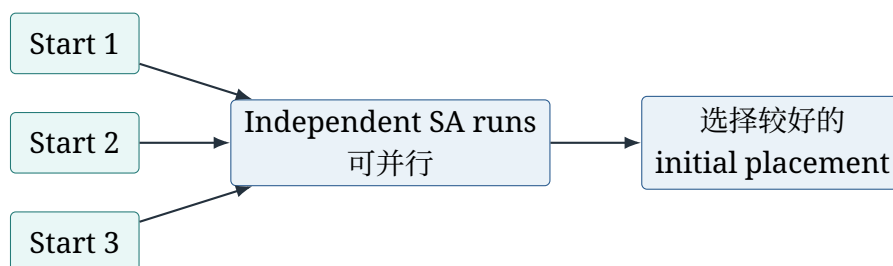
假设 $N_{Csize} = 100$ ，两个相同大小的 bins，先忽略 $W_B + H_B$ 的共同尺度：

状态	bin 内 cluster 总 size	平方项	相对拥挤惩罚
A	20	$20^2 = 400$	低
B	40	$40^2 = 1600$	约为 A 的 4 倍
C	60	$60^2 = 3600$	约为 A 的 9 倍

所以 SA 不会只顾着缩短 HPWL，把所有 clusters 都挤到“连线中心”。

6 为什么 SA 要多次随机启动？

论文让 SA 从多个不同随机起点独立运行，扩大搜索空间，并指出这些 runs 可以并行。（论文 Sec. III-A）



这一步的本质不是“SA 比 QP 更精确”，而是利用随机搜索的全局跳跃能力，降低 QP 从差的 basin 开始的概率。

到这里你应该能回答：为什么 **AMF-Placer** 不是直接对所有 **instances** 做 **SA**？

答案：因为规模太大。它先 partition 成 clusters，把 SA 放在粗粒度空间里；后面再用 analytical QP 处理细粒度连续坐标。

7 进入 Quadratic Placement：为什么不直接最小化 HPWL？

论文写出标准 HPWL 目标：

$$W(\mathbf{x}, \mathbf{y}) = \sum_{e \in E} \left[\max_{i,j \in e} |x_i - x_j| + \max_{i,j \in e} |y_i - y_j| \right]. \quad (3)$$

(论文 Eq. (3))

从优化角度看，它包含 max 和绝对值，在极值点切换处不可微/不光滑；直接塞进标准 quadratic solver 并不合适。

AMF-Placer 因此沿用 FPGA analytical placement 中常见的 **Bound2Bound (B2B)** weighted quadratic model，用二次项近似 net 的 HPWL：

$$\widetilde{W}_e = \sum_{i,j \in e} [w_{x,ij}^{B2B} (x_i - x_j)^2 + w_{y,ij}^{B2B} (y_i - y_j)^2]. \quad (4)$$

(论文 Eq. (4))

记住一个核心变化：

$$|x_i - x_j| \implies w_{ij} (x_i - x_j)^2.$$

变成二次形式后，整个 placement 可以写成稀疏线性/二次系统，求解效率远高于直接处理离散合法位置。

论文原意

论文在这一页没有给出 $w_{x,ij}^{B2B}, w_{y,ij}^{B2B}$ 的具体计算公式，而是引用 Bound2Bound net model [23]。因此本笔记今天只解释“权重为什么必须随 placement 更新”，不虚构论文未给出的 B2B 权重细节。

8 QP 里的真正难点：合法位置是离散约束

理想目标写成：

$$\min_{\mathbf{x}, \mathbf{y}} \sum_{e \in E} \widetilde{W}_e, \quad \text{s.t. } (x_i, y_i) \text{ 对 instance } v_i \text{ 是合法位置.} \quad (5)$$

(论文 Eq. (5))

问题在于：

- QP 最喜欢连续变量；
- FPGA legal sites 是离散的；
- 不同 instance type 只能去特定资源；
- macro 甚至需要多个相邻 sites。

如果每一步 QP 都强行满足这些离散约束，问题会变得难求。AMF-Placer 的关键处理是：**不在 QP 内直接做完整 legalization，而是加入 anchors + pseudo nets，把“希望靠近合法位置”变成软约束。**

这就是 analytical placement 经常采用的思路：**先把离散可行性松弛成连续 penalty，快速找到低线长位置；再通过 spreading/legalization 逐步把解拉回可实现区域。**

9 Pseudo net + anchor：把硬约束变成“弹簧”

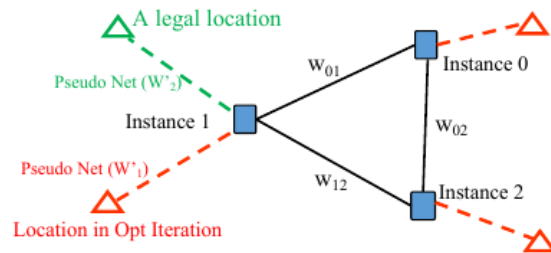


图 2 原论文 Fig. 4 的主要图像：蓝色方块是 instances，三角形是 anchors，虚线是 pseudo nets。红色 anchor 对应前一轮优化位置；绿色 anchor 示意潜在合法位置。

最终 QP 被写成：

$$\min_{x,y} \sum_{e \in E} \widetilde{W}_e + \sum_{e_p \in E_p} w_{e_p} \widetilde{W}_{e_p}. \quad (6)$$

（论文 Eq. (6)）

第一项是真实 nets 的线长近似；第二项是 pseudo nets 的 penalty。

9.1 把 pseudo net 想成弹簧



如果 w_{e_p} 很小，弹簧软，instance 更愿意听真实 nets 的“拉力”；如果 w_{e_p} 很大，弹簧硬，instance 会更靠近 anchor。

只看一维、只看一个真实目标与一个 anchor，可写成一个帮助理解的最小模型：

$$J(x) = k(x - p)^2 + \lambda(x - a)^2.$$

令导数为零：

$$2k(x - p) + 2\lambda(x - a) = 0,$$

得到

$$x^* = \frac{kp + \lambda a}{k + \lambda}.$$

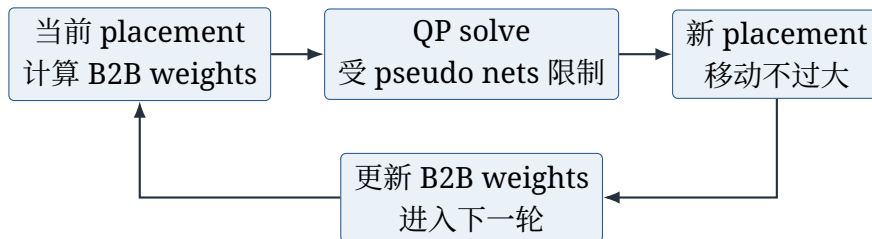
所以 anchor 的权重 λ 越大，最优位置越靠近 a 。这不是论文新增公式，只是把 pseudo net 的“引

力”用最简单的数学形式解释出来。

10 Pseudo net 为什么还有第二个作用：限制单次移动

B2B 权重 $w_{x,i_j}^{B2B}, w_{y,i_j}^{B2B}$ 是根据当前 **placement** 设置的。论文明确指出：如果一次 QP 之后 placement 变化太大，原来的 **weighted quadratic model** 可能会很差地近似真实 HPWL。（论文 Sec. III-B）

因此 **pseudo net** 不只是为了 legalization，它还像一个 **trust region / stabilization mechanism**：让每轮 instance 不要跳得太远，随后更新 B2B 权重，再做下一轮 QP。



这点很重要：AMF-Placer 的 QP 不是“一次求解完”。它是**迭代重加权**：当前位置决定 B2B 权重，QP 更新位置，随后重新估计权重。

11 论文最值得记住的改动：pseudo-net weight 乘上 pinNum

以前的相关方法采用：

$$w_{e_p, v_i} = \frac{\alpha}{\text{movement}(v_i)}. \quad (7)$$

（论文 Eq. (7)）

AMF-Placer 改为：

$$w_{e_p, v_i} = \frac{\alpha}{\text{movement}(v_i)} \times \text{pinNum}(v_i) \quad (8)$$

（论文 Eq. (8)）

11.1 先逐项理解

- α ：全局 pseudo-net strength，随着 global placement 推进而逐步增强；
- $\text{movement}(v_i)$ ：上一轮 instance 的移动距离；
- $\text{pinNum}(v_i)$ ：对应 instance 的外部 pin 数量。

旧式权重里，如果上一轮 movement 小，则下一轮权重大，instance 会更稳定；如果 movement 大，权重相对小，允许它继续调整。

AMF-Placer 再乘 pinNum ，是因为 macro 往往连接极多外部 nets。论文给出的 CARRY macro 例子甚至可能连接超过 200 条外部 nets。这样一个高连接密度 macro 如果在一次 QP 中移动很远，会同时改变大量 nets 的几何关系，使基于旧 placement 生成的二次近似迅速失真。（论文 Sec. I-B, Sec. III-B）

所以 Eq. (8) 的直觉不是“pin 多，所以更重要”这么简单，而是：

pin 多 $\Rightarrow$ 牵动的 nets 多 $\Rightarrow$ 大位移造成的模型失真更严重 $\Rightarrow$ 应移动得更保守

12 用一个数字例子真正看懂 Eq. (8)

假设两类 instance 在上一轮都移动了 2 个单位，且当前 $\alpha = 1$ ：

对象	movement	pinNum	AMF-Placer pseudo weight
普通 LUT/FF 小对象	2	4	$1/2 \times 4 = 2$
大型 macro	2	80	$1/2 \times 80 = 40$

macro 的 pseudo-net weight 是 40，而小对象只有 2。也就是说，当真实 nets 想把它们拉向别处时：

- 小对象仍然可以比较灵活地重新定位；
- 高 pin-count macro 会被更强地“拴”在当前/潜在合法位置附近。

这是一种 **connection-density-aware damping**。它不是冻结 macro，而是让 macro 的移动步子更小、更稳定，从而降低 weighted quadratic model 在宏密集设计中的剧烈失真。

论文报告的消融实验把这一改动称为 Tech2(interconnection-density-aware pseudo net weight)；关闭 Tech2 后，不少 benchmark 的 HPWL 与 runtime 都会恶化。真正的实验数据会在后续实验日统一学习。（论文 Sec. IV-B）

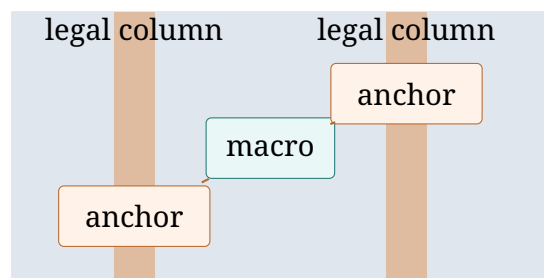
13 还有一种 pseudo net：连向“潜在合法位置”

前面讲的是连接到上一轮位置的 pseudo net。AMF-Placer 还专门为 mixed-size macro 加入另一类 pseudo nets：把 macro 连接到若干 **potential legal locations**。（论文 Sec. III-B）

原因是 FPGA 上大型 macro 的合法位置非常稀疏：

- DSP macro 只能去 DSP columns；
- BRAM macro 只能去 BRAM columns；
- 有的 macro 需要连续的多个 sites；
- 多个 macro 之间还不能重叠。

如果 QP 只最小化 wirelength，可能把 macro 推到一个“线长很好但离任何合法位置都很远”的连续坐标。最后 legalization 再把它硬搬到合法 site，HPWL 会突然恶化。



这些 legalization pseudo nets 提前把“未来要合法化”的信息注入 QP，使 wirelength optimization 与 legalization 不再完全割裂。具体怎样寻找候选位置、何时从 rough legalization 转到 exact legalization，是 Day 4 的 Progressive Macro Legalization 重点。

14 把 Quadratic Placement 写成矩阵，你会更容易理解求解器

对很多形如 $w_{ij}(x_i - x_j)^2$ 的项求和，可以整理成

$$\frac{1}{2} \mathbf{x}^T Q_x \mathbf{x} - \mathbf{b}_x^T \mathbf{x} + \text{const},$$

y 方向同理。pseudo-net 到固定 anchor 的项会贡献到矩阵对角和右端项。于是每一轮 QP 的核心就接近求一个稀疏线性系统：

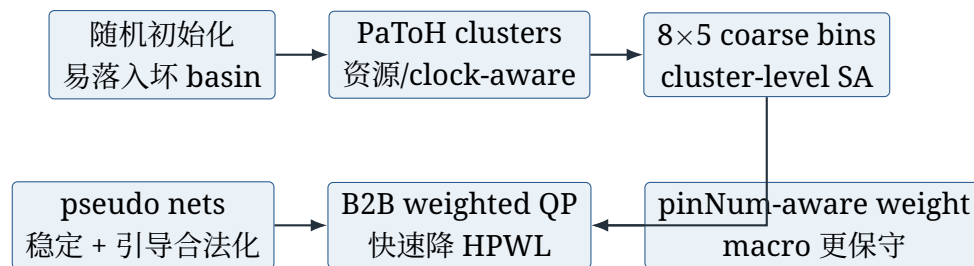
$$Q_x \mathbf{x} = \mathbf{b}_x, \quad Q_y \mathbf{y} = \mathbf{b}_y.$$

这是理解 quadratic placer 的标准数学视角；论文正文没有把 Eq. (6) 展开成这个矩阵形式，但它与二次目标等价。

论文最终使用 Eigen3 solver，并基于 OpenMP 利用并行性。（论文 Sec. III-B）

因此 QP 快的根本原因不是“公式看起来简单”，而是整个目标被组织成结构化、稀疏的二次形式；相比 combinational legalization，它更适合数值线性代数求解。

15 Day 2 总结：只记住这张因果图



如果今天真正学懂，你应该能自己解释下面 5 句话：

1. 初始 placement 对现代 heterogeneous FPGA 并非无关紧要；
2. AMF-Placer 先做 cluster-level SA，是在质量与 runtime 之间折中；
3. HPWL 不适合直接作为 quadratic solver 的目标，因此用 B2B weighted quadratic approximation；
4. pseudo nets 同时承担“限制单轮移动”和“提前引入 legalization 信息”两种职责；
5. pin-count multiplier 让高连接密度 macro 更稳定，避免其大幅移动导致 wirelength model 严重失真。

下一天预告：Cell Spreading

Day 3 会进入论文最有 FPGA mixed-size 特色的另一部分：**Cell Spreading**。重点包括：macro 为什么会制造 spreading deadlock、resource supply fluctuation 为什么能“挤出”实例、utilization-guided

spreading window 如何避免过度扩张, 以及 forgetting rate

$$\gamma = 1 - 0.95 \frac{HPW L_{lower}}{HPW L_{upper}}$$

为什么能在收敛后期抑制 spreading 对线长的破坏。